

CO 2 – ASSESMENT TOOL 1

**JAVA STRING HANDLING AND COLLECTION
FRAMEWORK**

NAME: P.V.N. ANISH BABU

REG NO: 192472329

COURSE CODE: CSA0924

COURSE NAME: JAVA PROGRAMMING

SLOT: D

Program Analysis: Producer Extends, Consumer Super (PECS)

AIM

To determine suitable wildcard bounds for a method that copies elements from a source list to a destination list using the **Producer Extends, Consumer Super (PECS)** principle and justify the design.

Theory

The **PECS** principle states:

- **Producer Extends (? extends T)**: Use when a collection produces data.
- **Consumer Super (? super T)**: Use when a collection consumes data.

PSEUDO CODE

START

Create a generic method copy()

Input:

Source list of type ? extends T

Destination list of type ? super T

For each element in source list

Read element from source

Add element to destination

End For

Display destination list

STOP

JAVA CODE

```
import java.util.ArrayList;
import java.util.List;

public class PECSExample {

    public static <T> void copy(
        List<? extends T> source,
        List<? super T> destination) {

        for (T element : source) {
            destination.add(element);
        }
    }

    public static void main(String[] args) {

        List<Integer> source = new ArrayList<>();
        source.add(10);
        source.add(20);
        source.add(30);
        List<Number> destination = new ArrayList<>();
        copy(source, destination);
        System.out.println("Source List    : " + source);
        System.out.println("Destination List : " + destination);
    }
}
```

OUTPUT

Source List: [10, 20, 30]

Destination List: [10, 20, 30]

Professional Analysis

Method Signature

```
public static <T> void copy(  
    List<? extends T> source,  
    List<? super T> destination)
```

Analysis

Component	Wildcard Bound	Reason
Source List	? extends T	Source produces elements
Destination List	? super T	Destination consumes elements

PECS Rule

- **Producer** → **Extends**
- **Consumer** → **Super**

Advantages

1. Ensures compile-time type safety.
2. Supports inheritance and polymorphism.
3. Prevents invalid insertions into the source list.
4. Makes the method reusable for different data types.
5. Follows Java generic best practices.

CONCLUSION

The wildcard bounds `List<? extends T>` for the source and `List<? super T>` for the destination correctly implement the PECS principle. This design maximizes flexibility while maintaining type safety in generic copy operations.